

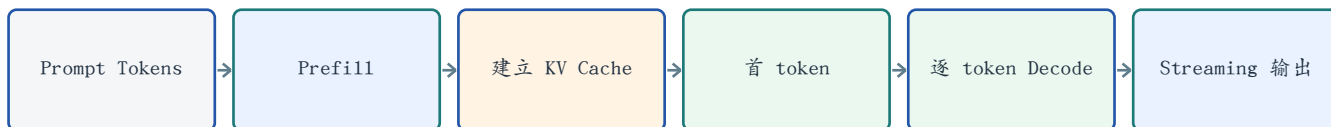
第22讲

KV Cache、Prefill、Decode 与推理性能

为什么长对话越来越贵？为什么 GQA / MQA 会成为现代 LLM 的重要设计？

本讲主线

一次 LLM 推理可以拆成 Prefill 与 Decode 两个阶段；KV Cache 用显存换重复计算；上下文越长、并发越高，缓存和调度问题越重要。



第19讲讲“流程”，本讲进一步看“系统代价”

学习定位：以后看到 TTFT、TPOT、throughput、continuous batching、GQA、MQA、PagedAttention，不再把它们当作孤立术语。

1. 先把第19讲压缩成两个阶段：Prefill 与 Decode

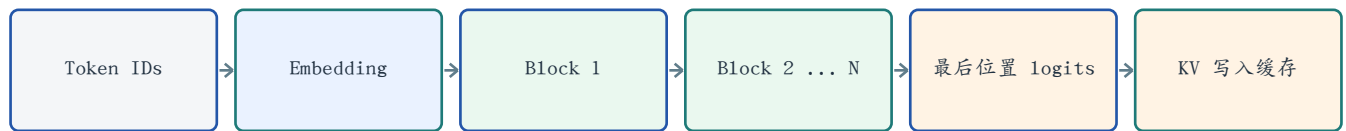
一次典型 Decoder-only LLM 推理，不是从头到尾都以同一种计算形态运行。最重要的分界是：Prefill（处理已有输入）与Decode（逐步生成新 token）。

阶段	输入形态	并行性	主要结果
Prefill	一次处理整个 prompt 的很多 token	token 维度上高度并行	得到首个 next-token 分布，并建立历史 K/V
Decode	每一步只新增 1 个 token	跨时间步天然串行	读取历史 cache，生成下一个 token

因此，一个模型“回答慢”时必须继续问：是首 token 等得久，还是后续 token 吐得慢？

2. Prefill 到底在做什么？

Prefill 可以理解成：模型先把当前 prompt 整体“读进去”。对每一层 Attention，它会为 prompt 中各 token 计算 Q、K、V，并完成因果注意力。



虽然模型是 causal 的，但 Prefill 并不是“真的一个 token 一个 token 慢慢读”。训练和 prefill 时，已知的 prompt token 可以作为一个矩阵批量送入 GPU。

直觉

Prefill 更像“把一本已经写好的材料整批读入”；Decode 才像“每写一个新字，都要根据前面的内容再决定下一个字”。

3. Decode 为什么天然更串行?

第 $t+1$ 个生成 token 依赖第 t 个 token

已经被选出来并追加到上下文，所以时间轴上存在不可绕过的依赖链。



这也是为什么长回答的总时间通常会随输出 token 数明显增加。GPU 可以并行服务多个请求，但单条序列自己的 decode 时间步仍然是递归推进的。

4. KV Cache 到底缓存的是什么？

在 Self-Attention 中，每个旧 token 的 Key 和 Value 在后续 decode 步骤中会被重复使用。如果每生成一个新 token 都把所有旧 token 的 K/V 重新算一遍，会造成大量重复工作。

所以系统保存：

past K, past V -> KV Cache

新 token 到来时，只需计算它自己的 K/V，再与历史 K/V 拼接或写入缓存结构。

不是“记忆数据库”

KV Cache 是本次推理计算过程中的中间张量缓存。它服务的是高效 Attention，不等价于 Agent Memory，也不等价于用户跨会话记忆。

5. 为什么缓存 K/V，而不是把 Q 也长期缓存？

当前新 token 的 Query 用来“向过去询问信息”。每一步只需要当前 Query 去匹配历史 Keys，并用得到的权重读取历史 Values。过去 token 的旧 Query 通常不会在未来步骤再次作为“当前查询”使用。

张量	未来 decode 中是否反复需要	直觉
历史 K	需要	旧 token 提供“可被匹配的索引”
历史 V	需要	旧 token 提供“被读取的内容”
历史 Q	通常不需要长期保存	旧问题已经问过了；下一步会有新的 Query

所以名称叫 KV Cache，而不是 QKV Cache。

6. KV Cache 的大小怎么估？

对标准 Decoder Transformer，可以先记住一个非常实用的简化数量级公式：

$$\text{KV bytes} \approx \text{batch} \times \text{seq_len} \times \text{layers} \times 2(K,V) \times \text{kv_heads} \times \text{head_dim} \times \text{bytes_per_element}$$

它告诉你五件事：

上下文 seq_len 翻倍，KV Cache 大致线性翻倍；

batch / 并发序列增加，缓存也近似成比例增加；

层数越多，缓存越大；

KV head 数越多，缓存越大；

BF16/FP16、FP8、量化 KV 等不同精度会改变每个元素的字节数。

注意

这是理解数量级的简化式。真实推理框架还会有元数据、内存碎片、对齐、分页、其他激活和模型权重等开销。

7. 手算一个例子：为什么 32K 长上下文很吃显存？

假设一个教学配置：32 层、32 个 attention heads、head_dim=128、KV 用 2 bytes 元素、batch=1。

如果使用普通 MHA，kv_heads=32。对于 4096 token：

$$2 \times 32 \text{ layers} \times 4096 \times 32 \text{ KV heads} \times 128 \times 2 \text{ bytes} = 2 \text{ GiB}$$

把长度扩大到 32K（8 倍），缓存约变成 16 GiB。

关键

这里甚至还没算模型权重、临时激活、CUDA workspace、batch 并发等。长上下文很容易把“推理瓶颈”从纯计算推向显存容量和内存带宽。

8. MHA、GQA、MQA：为什么要减少 KV Heads?

结构	Query Heads	KV Heads	核心理念
MHA	多	通常与 Q heads 相同	每个 Q head 拥有自己的 K/V head
GQA	多	较少	一组 Q heads 共享一组 K/V
MQA	多	1	所有 Q heads 共享同一 K/V head

如果 Q heads 仍是 32，而 KV heads 从 32 降到 8，那么 KV Cache 大致变为原来的 1/4。降到 1，则大致变为 1/32。

这就是 GQA / MQA 对推理系统极有吸引力的原因之一：减少 K/V 存储和读取压力。

9. 同一个 4K Context：MHA、GQA、MQA 的缓存差多少？

结构	KV Heads	教学例子的 4K KV Cache	相对 MHA
MHA	32	约 2.000 GiB	1×
GQA	8	约 0.500 GiB	1/4
MQA	1	约 0.0625 GiB	1/32

如果扩展到 32K，则在同一简化配置下大致变成 16 GiB、4 GiB、0.5 GiB。

不要误解

GQA/MQA 并不是“把 Attention Head 全部删掉”。Query heads 仍可很多；变化主要在 K/V 的共享方式。能力、训练方式和推理效率之间需要折中。

10. TTFT：为什么长 Prompt 让“第一句话”等更久？

TTFT (Time To First Token)：从请求开始，到第一个输出 token 可用所经历的时间。

简化地看：

$$\text{TTFT} \approx \text{queue} + \text{request overhead} + \text{prefill} + \text{first-token sampling}$$

Prompt 越长，Prefill 通常越重，所以 TTFT 往往会上升。

影响因素	如何影响 TTFT
Prompt 长度	更长输入需要更多 prefill 计算
服务器排队	高并发时请求先等待 GPU 调度
Batching	可能提高总体吞吐，但单请求可能需要等候拼批
Prefix Cache	若大量前缀可复用，可能减少重复 prefill
硬件/内核	矩阵吞吐、内存带宽、Attention kernel 都会影响

11. TPOT /

ITL：为什么“开头快，后面吐字慢”是另一种问题？

TPOT (Time Per Output Token) 或 ITL (Inter-Token Latency) 反映 Decode 阶段连续两个输出 token 之间的时间。

粗略地说：

$$\text{Generation time after first token} \approx (\text{output_tokens} - 1) \times \text{TPOT}$$

因此一个 500 token 的长回答，即使 TTFT 很小，如果 TPOT 较高，完整答案仍可能花很久。

用户体验

流式输出改善的是“可感知等待”：用户先看到首 token；它并没有自动降低每个后续 token 的计算成本。

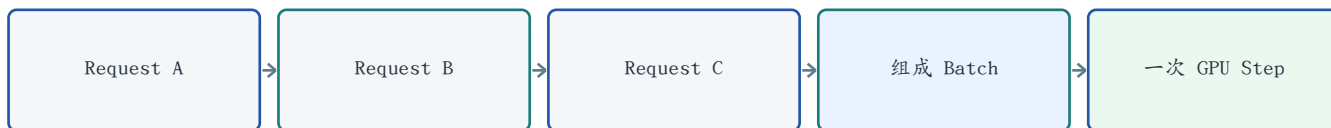
12. Latency 与 Throughput：不是同一个指标

指标	关心的问题	典型单位
Latency	一个请求多久完成	ms / s per request
TTFT	多久看见第一个 token	ms
TPOT / ITL	后续 token 吐多快	ms/token
Throughput	服务器单位时间总共处理多少	tokens/s 或 requests/s
Concurrency	同时服务多少活跃请求	requests / sequences

一个服务系统可能通过更大的 batch 提高 throughput，但某些请求的等待时间也可能上升。因此“吞吐最大”和“单用户延迟最低”经常不是同一个优化点。

13. 为什么要 Batching?

GPU 最擅长大块并行矩阵运算。如果一次只服务一个很短的请求，很多计算单元可能吃不满。Batching 的思路是把多个请求一起送入 GPU。



Batching

可以提高硬件利用率和总吞吐，但会带来新的问题：请求长度不同、到达时间不同、结束时间不同。

14. Static Batching 的“短请求陪跑”问题

假设 4 个请求需要分别生成：12、34、7、40 个 token。最朴素的固定 batch 可能需要一直运行到最长的 40 步。

$$\text{toy padded slots} = \text{batch_size} \times \text{max_output_length} = 4 \times 40 = 160$$

真正有用的 token 只有：

$$12 + 34 + 7 + 40 = 93$$

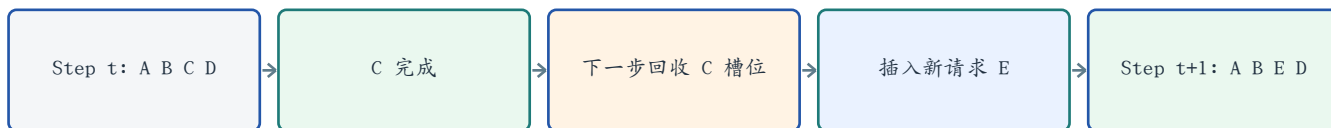
教学性的利用率只有 $93/160 \approx 58\%$ 。

现实更复杂

现代框架会 mask 已结束序列、回收资源并做更复杂调度，所以不能把这个 toy utilization 当成真实 GPU 利用率；它只是说明“序列长度不齐会造成调度浪费”。

15. Continuous Batching: 为什么可以边生成边换人?

Continuous Batching (连续批处理) 的核心思想是: 不必等整个 batch 全部完成才接下一批。某个请求结束后, 可以把新请求插入后续 decode step。



这样更适合在线 LLM 服务, 因为用户请求是持续到达、长度各异的。

它优化的是服务器调度, 而不是改变 Transformer 数学本身。

16. 为什么 Decode 常常更受 Memory Bandwidth 约束？

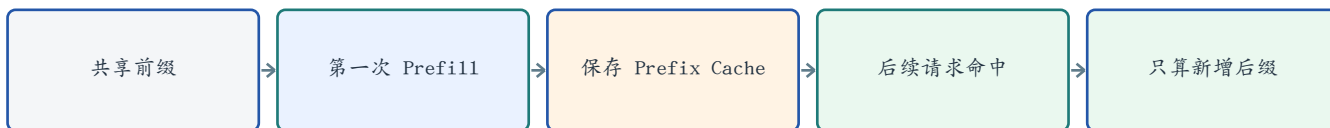
Prefill 可以构造很大的矩阵运算，往往更容易利用 GPU 的算力；Decode 每步只有少量新 token，却需要反复读取模型权重和越来越大的 KV Cache。

阶段	常见系统直觉
Prefill	更偏大矩阵计算；长 prompt 时 compute 很重
Decode	每步工作量较小但高度重复；模型权重 / KV 读取和内存带宽常很关键

这也是为什么“训练 GPU 很强”并不自动意味着“单 token 推理延迟一定最低”；推理系统还高度依赖内存层级、kernel 和调度。

17. Prefix Caching: 重复 System Prompt 为什么可能省下来?

很多请求拥有完全相同的长前缀，例如固定 System Prompt、工具定义、共享文档。如果服务器能够识别相同前缀，就可能复用那一部分已经计算好的中间缓存。



这可以降低重复 prefill，但需要满足前缀一致、缓存仍有效、服务端支持等条件。

Agent 场景

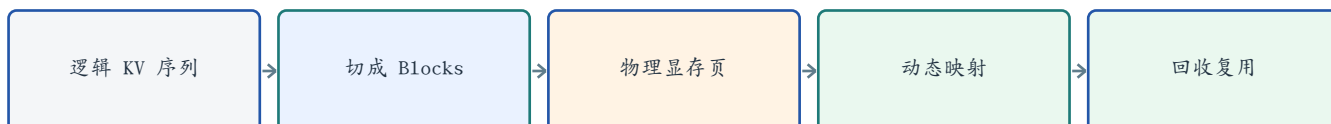
工具 schema 很长、多个 Agent 使用同一基础规则时，prefix caching 可能特别有价值；但具体 API 是否给你复用、如何计费，需要看服务实现。

18. Paged KV /

PagedAttention：为什么“显存管理”也会成为论文问题？

不同请求的上下文长度和生成长度都不一样。如果为每条序列提前分配一大块连续 KV 内存，容易产生浪费和碎片。

PagedAttention 一类系统思想借鉴虚拟内存分页：把 KV Cache 切成较小块，按需分配和映射。



核心不是改变 Attention 公式，而是更高效地管理“缓存放在哪里”。

系统科学视角

同一个神经网络，因为内存管理、调度和缓存策略不同，最终可服务的并发、延迟和成本可能差很多。这就是 AI Systems 研究的重要性。

19. KV Cache 也能量化吗？

既然 KV Cache 会占大量显存，一个自然办法就是用更低精度存储 K/V，例如从 BF16/FP16 降到更低位宽。

做法	潜在收益	潜在风险
降低 KV 精度	减少显存、带宽压力	量化误差可能影响长上下文与生成质量
减少 KV heads	结构性降低缓存体积	需要模型架构/训练兼容
缩短/压缩 Context	直接减少 seq_len	可能丢掉重要信息
Eviction / Sliding Window	只保留部分历史	远距离信息可能不可访问

所以“减少 KV Cache”不是只有一种技术路线。

20. 长 Context 为什么会同时打击 TTFT 和显存？

把第21讲和本讲合起来：上下文变长会至少产生两类压力。

压力	机制	结果
Prefill 压力	需要处理更多输入 token	TTFT 上升、计算量增加
KV 压力	每层保存更多历史 K/V	显存占用与内存流量上升
并发压力	每个请求都占更多 KV	同一 GPU 可同时服务的序列数下降
成本压力	输入 token 和运行时间增加	API / 自托管成本上升

所以“支持 1M context”首先是容量声明；真正的产品体验还取决于 prefill 速度、cache 管理、长上下文可靠性和成本。

21. 并发为什么会让 KV Cache 问题突然严重？

单个请求 0.5 GiB KV 似乎不多，但如果同一 GPU 同时有 40 条活跃序列，理论缓存需求就可能接近 20 GiB。

$$\text{total KV memory} \approx \text{per-sequence KV} \times \text{active sequences}$$

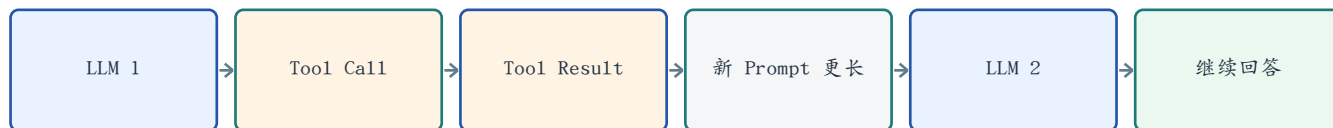
而 GPU 还必须同时放模型权重、临时 workspace、框架开销等。

服务系统的真实问题

在线推理优化常常不是“能不能跑一条请求”，而是“在延迟目标下，能同时稳定服务多少请求”。

22. Tool Use 为什么会不断制造新的 Prefix?

一个 Agent 调用工具时，模型通常先生成 tool call; Runtime 执行工具; 然后把 tool result 加入上下文，再次调用模型。



这意味着一次“Agent 任务”可能包含多个独立或增量的 prefix/decode episode。工具返回越长，下一轮 prompt 越长。

因此工具输出压缩、结构化字段选择、本地状态管理都会直接影响成本。

23. 多 Agent 系统：KV Cache 和上下文成本会怎样放大？

如果 5 个 Agent 各自维护 16K context，它们不是共享一个“免费的大脑工作台”。每个 Agent 的调用都有自己的上下文、KV 状态和生成成本。

设计	可能的系统代价
全量广播所有消息	每个 Agent prompt 越来越长；大量重复 token
每个 Agent 独立长记忆	总缓存 / 检索成本随 Agent 数增长
多轮讨论	反复产生 prefill + decode 调用
中央汇总器	额外一次或多次长 prompt
选择性通信	可能降低 token 和缓存压力，但需要路由机制
科研问题	通信拓扑、上下文压缩、消息选择和共享 memory，不只是“工程省钱”，也可能改变信息传播、错误相关性和集体行为。

24. 做 Agent Systems 实验时，至少报告哪些推理指标？

如果只报告 Accuracy，很难判断一个多 Agent 方法到底是“算法更聪明”还是“用了更多推理资源”。

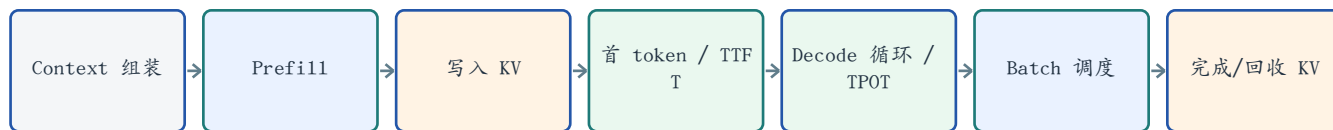
类别	建议记录
调用规模	model calls / agent calls / turns
Token	input tokens / output tokens / cached tokens (若可得)
延迟	TTFT、TPOT 或总 wall-clock latency
工具	tool calls、tool result size
并发	batch size / concurrency / hardware or API setup
成本	总 API 成本或 GPU-hours
质量	accuracy / success / verifier score + 多随机种子

这样才有机会做真正的 compute-matched / cost-matched comparison。

25. 常见误区

误区	纠正
KV Cache = 长期记忆	KV Cache 是当前推理的 Attention 中间张量缓存
Prefill 也必须逐 token 串行	已知 prompt 可以矩阵化并行处理
GQA 就是少几个 Attention Heads	通常是减少 K/V heads, 让多个 Q heads 共享 K/V
吞吐越高, 单用户延迟一定越低	吞吐与 latency 存在调度权衡
长上下文只会增加 API token 费	还会影响 prefill、KV cache、并发和延迟
Agent 只调用一次 LLM	真实 Agent 往往是多次 inference episode

26. 一个完整的系统心智模型



模型数学 + 内存管理 + 调度策略共同决定最终体验

可以把现代 LLM Serving 理解成三个层次：

模型层：Transformer、GQA/MQA、量化等；

缓存/内存层：KV Cache、paged allocation、prefix reuse；

调度层：batching、continuous batching、请求优先级与并发。

同一个模型权重，换一个 Serving Stack，性能就可能明显不同。

27. 小实验：亲手算 KV Cache 与推理指标

材料包中的 04_kv_cache_inference_performance_demo.py 不需要 GPU，只用 NumPy / Matplotlib 做三个教学实验：

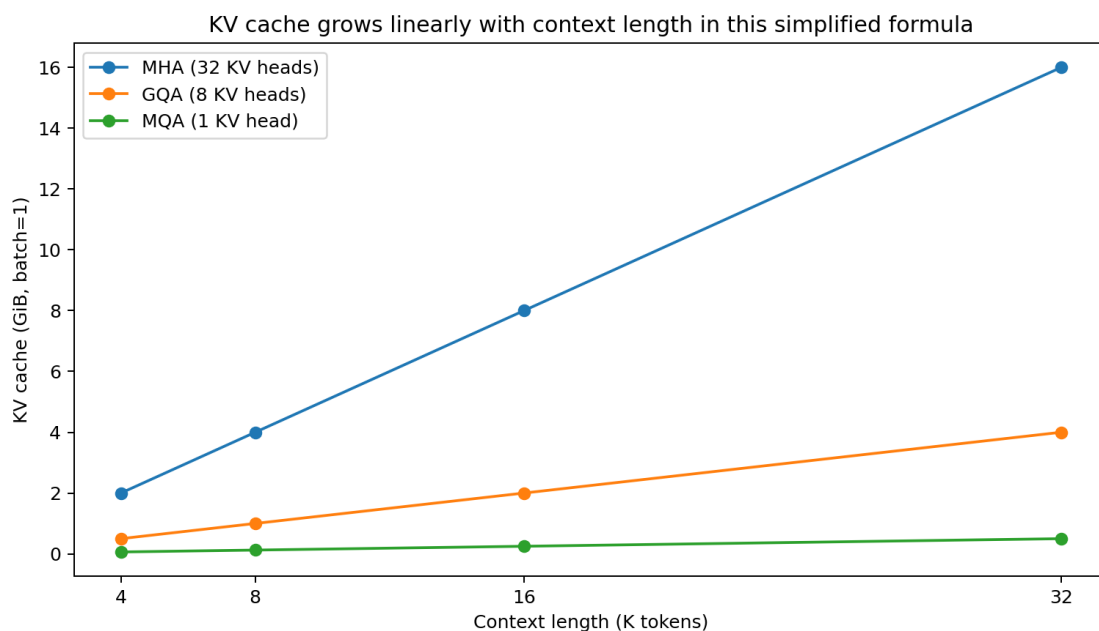
计算 4K / 8K / 16K / 32K context 下 MHA、GQA、MQA 的 KV Cache 大小；

用明确标注为“toy”的公式演示 prompt 变长时 TTFT 为什么会上升；

用 4 条不同长度输出说明 static batching 中的 padding / idle waste。

实际配置：32 layers、32 Q heads、head_dim=128、2 bytes/element、batch=1。

你应该重点观察：KV Cache 对 seq_len 是近似线性，对 kv_heads 也是近似线性。



28. 本讲总结：把术语压成 8 句话

Prefill: 一次把已有 prompt 批量读进去。

Decode: 按时间步逐 token 生成。

KV Cache: 保存历史 K/V, 避免后续重复计算。

TTFT: 多久看见第一个 token, 长 prompt 通常主要拉高这部分。

TPOT: 后续每个 token 的间隔, 决定长回答吐字速度。

GQA/MQA: 减少 KV heads, 共享 K/V, 从而降低缓存和带宽压力。

Continuous Batching: 请求完成就动态补新请求, 提高在线服务利用率。

Agent Systems: 多轮、多 Agent、工具调用会把上述所有成本成倍叠加。

最终心智模型：

Context 决定“要处理多少历史”；KV Cache 决定“如何少重复算”；Serving Scheduler 决定“如何让很多请求一起高效跑”。

29. 与下一讲的连接

现在你已经理解了：为什么长上下文不仅是“模型能不能看见”，还涉及显存、带宽、延迟和并发。下一讲我们进入模型“变轻”的另一条路线：

第23讲：量化、蒸馏与 MoE——为什么 70B

模型可以用更少显存运行？为什么“总参数”不等于“每个 token 都参与计算”？

下一讲会把 FP32 / BF16 / FP16 / INT8 / INT4、Weight Quantization、Activation、KV Quantization、Distillation，以及 Dense vs Mixture-of-Experts 统一到“模型部署成本”这一条主线。